第 3 章推導解答

DS03-E01 • 環狀回繞

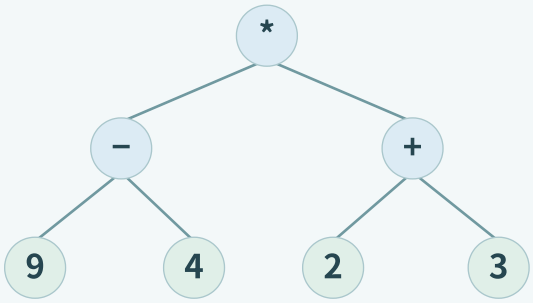
初始 $(head, size) = (0, 0)$。表中「—」表示沒有有效元素；若底層仍殘留舊字值，也不屬於抽象序列。enqueue 的槽為 $(head + size) \bmod 4$，dequeue 後 $head = (head + 1) \bmod 4$。

已完成操作	head	size	槽0	槽1	槽2	槽3	邏輯序列
初始	0	0	—	—	—	—	空
enqueue A	0	1	A	—	—	—	A
enqueue B	0	2	A	B	—	—	A,B
enqueue C	0	3	A	B	C	—	A,B,C
dequeue A	1	2	—	B	C	—	B,C
dequeue B	2	1	—	—	C	—	C
enqueue D	2	2	—	—	C	D	C,D
enqueue E	2	3	E	—	C	D	C,D,E
enqueue F	2	4	E	F	C	D	C,D,E,F
dequeue C	3	3	E	F	—	D	D,E,F
dequeue D	0	2	E	F	—	—	E,F
dequeue E	1	1	—	F	—	—	F

滿時 size=4；空時 size=0。這個表示法不靠 head 與某個 tail 索引相等來區分空滿。

DS03-E02 • 運算式轉換

樹根為 *，左子樹根 -、葉 9 與 4，右子樹根 +、葉 2 與 3。



後序求值：先pop右子式值，再pop左子式值

前序：根 → 左 → 右

* − 9 4 + 2 3

後序：左 → 右 → 根

9 4 − 2 3 + *

(9−4) × (2+3) = 5 × 5 = 25

前序為 `* - 9 4 + 2 3`，後序為 `9 4 - 2 3 + *`。後序堆疊依序為 `[9]`、`[9,4]`、`[5]`、`[5,2]`、`[5,2,3]`、`[5,5]`、`[25]`，即先算 9−4，再算 2+3，最後相乘。`8 3 -` 首先 pop 的 3 是右運算元，第二次的 8 是左運算元，因此得 5。交換會得 −5，違反原運算樹語義。

DS03-E03 • DFS 的第一條路徑

取有向邊 `s→a,a→b,b→c,c→t,s→t`，s 的鄰居依 a、t 排序。DFS 依序訪問 s、a、b、c、t，第一次抵達路徑長 4，parent 為 a←s、b←a、c←b、t←c。BFS 在處理 s 時已發現 a 與 t，t 的 parent=s、距離 1。若完整繼續，佇列訪問可為 s、a、t、b、c。反例只依賴 DFS 的鄰居次序，不否定 DFS 可達性正確性。

此 BFS 若完整執行，parent 為 a←s、t←s、b←a、c←b；距離依序 d(s)=0、d(a)=d(t)=1、d(b)=2、d(c)=3。父指標在首次發現時設定，後續遇到已發現的 t 不改成 c。

DS03-E04 • 語法與求值安全

`2 +` 在讀到二元 + 時堆疊不足兩值，立即拒絕。`2 3` 掃描完剩兩值，違反單一結果條件。`8 0 /` 取得右運算元 0 時拒絕零除。另需檢查整數剖析範圍、加減乘溢位，以及最小負整數除以 −1 的不可表示結果。若語法含一元負號，其 arity 必須由 token 類型確定，不能以當時堆疊大小猜測。

DS03-E05 • 掃描收尾與文法

讀入 token	運算子堆疊（底→頂）	輸出
9	∅	9
−	−	9
4	−	9 4
*	− *	9 4
2	− *	9 4 2
/	− /	9 4 2 *
1	− /	9 4 2 * 1
+	+	9 4 2 * 1 / −
3	+	9 4 2 * 1 / − 3
EOF	∅	9 4 2 * 1 / − 3 +

讀 + 時，先彈較高優先的 /，再彈同級左結合的 −。求值得 4×2=8、8/1=8、9−8=1、1+3=4。

輸入	拒絕點	原因
(9−4	EOF	彈−之後，仍有未配對左括號
9−4)	右括號	沒有左括號可配
9−	EOF	仍期待右運算元

右結合的 `2 3 2 ^ ^` 算 3^2=9，再算 2^9=512；左結合的 `2 3 ^ 2 ^` 算 2^3=8，再算 8^2=64。

DS03-E06 • 佇列擴容的抽象值

C 由舊槽2到新槽0，D 由3到1，E 由0到2，F 由1到3；提交後 head=0,size=4,capacity=8。直接複製舊四格並留head=2，邏輯第2項會讀新槽 `(2+2) mod 8=4`，它未建構而不是E，故抽象序列已錯。按邏輯順序搬動一次為Θ(n)，某次入列最壞Θ(n)；僅在滿時倍增且元素搬移有常數上界，總搬移形成幾何級數，從空的n次入列總Θ(n)、攤銷Θ(1)。若值移動成本不固定則需另乘該成本。固定容量最壞Θ(1)保證不能與這個攤銷保證混寫。

DS03-E07 • 兩個堆疊的混合操作

序列均由底寫到頂；初始 I=O=[]、Φ=0。實際成本只數 stack push/pop，不數常數次空檢查。

操作	I	O	FIFO	c	ΔΦ	c+ΔΦ
enqueue 4	[4]	[]	[4]	1	2	3
enqueue 7	[4,7]	[]	[4,7]	1	2	3
dequeue→4	[]	[7]	[7]	5	−4	1
enqueue 9	[9]	[7]	[7,9]	1	2	3
dequeue→7	[9]	[]	[9]	1	0	1
dequeue→9	[]	[]	[]	3	−2	1

第一次出列搬兩值，共2次pop、2次push，再pop4，因此成本5。第二次出列只從O取7，不搬I的9；最後才搬9。總實際成本12，總攤銷成本12，因為初末勢能皆0。一般前綴的最終勢能可能大於0，實際總成本只會小於或等於攤銷總成本。

DS03-E08 • 堆疊排列與所需容量

目標	操作／卡住的位置	判定
2,1,4,3	push1,push2,pop2,pop1,push3,push4,pop4,pop3	可行，最大深度2
3,1,4,2	push1,push2,push3,pop3；此時top=2，下一個要求1	不可行；push4只會再蓋住1

第一組至少要容量2，因為要在1尚未輸出時先輸出2，必須同時存1與2；所列操作達到此下界，所以最小容量確為2。

$C_0=1$、$C_1=1$、$C_2=2$、$C_3=5$，所以 $C_4=C_0C_3+C_1C_2+C_2C_1+C_3C_0=5+2+2+5=14$。4!=24計入了被LIFO阻擋的排列；上表第二組即是反例。

DS03-E09

`([])` 的堆疊依次為 `(`、`([`、`(`、空，因此接受。`([)]` 讀到索引 2 的 `)` 時，top 是 `[`，種類不符而拒絕。`(()` 掃描後尚有 `(`，在輸入結束時拒絕；空字串掃描結束即為空，接受。`([)]` 左右種類的數量都相同，卻交錯而非巢狀，故僅計數不夠。

DS03-E10

racecar 前半推入 r,a,c，跳過中央 e，依次比較並彈出 c,a,r，全部相同而接受。abca 推入 a,b 後，先比 c 與 top b，立即拒絕。每次成功比較消耗後半段的一個符號，因此最多 $\lfloor n/2 \rfloor$ 次；提早失配只會更少。單一碼點 é 必為碼點迴文，但其 UTF-8 bytes 為 C3 A9，兩個 byte 不相等，故不是 byte 迴文。

DS03-E11

aabb 接受；aab 結束時多一個 a；abb 的第二個 b 沒有可配的 a；abab 進入 b 階段後再出現 a；空字串因 $n \geq 1$ 而拒絕。若改成 $n \geq 0$，這五個輸入中只有空字串改為接受。

DS03-E12

head/size 最多 4 值，size=0 空、size=4 滿；預留一格最多 3 值，front=rear 空、(rear+1) mod 4=front 滿；加 tag 最多 4 值，索引相等且 tag=false 時空，索引相等且 tag=true 時滿。head 指首元素，其他兩種 front 指首元素之前；索引含義和空滿規則共同決定契約，不能只替換其中一部分。

參考資料與署名

Lin Cing，《資料結構》手寫筆記；C++ 標準草案 stack、queue；MIT 6.006，Lecture Notes 9–10。
Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。
https://github.com/xinhi6716/computer-science-self-study